

TouchDesigner上級編 2 - TouchDesigner + Shader (GLSL) 応用 GLSL MAT

今日の内容

- TouchDesignerでShader (GLSL) を試してみる、応用編
- GLSL MATを試してみる
- Fragment Shaderに加えて、Vertex Shaderも編集
- 3DオブジェクトにShaderを適用してみる

サンプルプログラム

1. GLSL TOP基本: 移動する波
2. GLSL TOP基本: 拡がる同心円
3. GLSL TOP: バンプマッピングとハイトマッピングで凸凹表現
4. GLSL TOP: 球面に平面的に波を描く
5. GLSL MAT: 球面に立体的に波を描く
6. GLSL MAT: 中心からの距離で色を変化させる
7. GLSL MAT: アルファ値を使ってピクセルの描画を制御する
8. Phong MATのShaderを編集してオリジナルの表現を作る - 球体
9. Phong MATのShaderを編集してオリジナルの表現を作る - グリッド

前回の復習 1: 移動する波

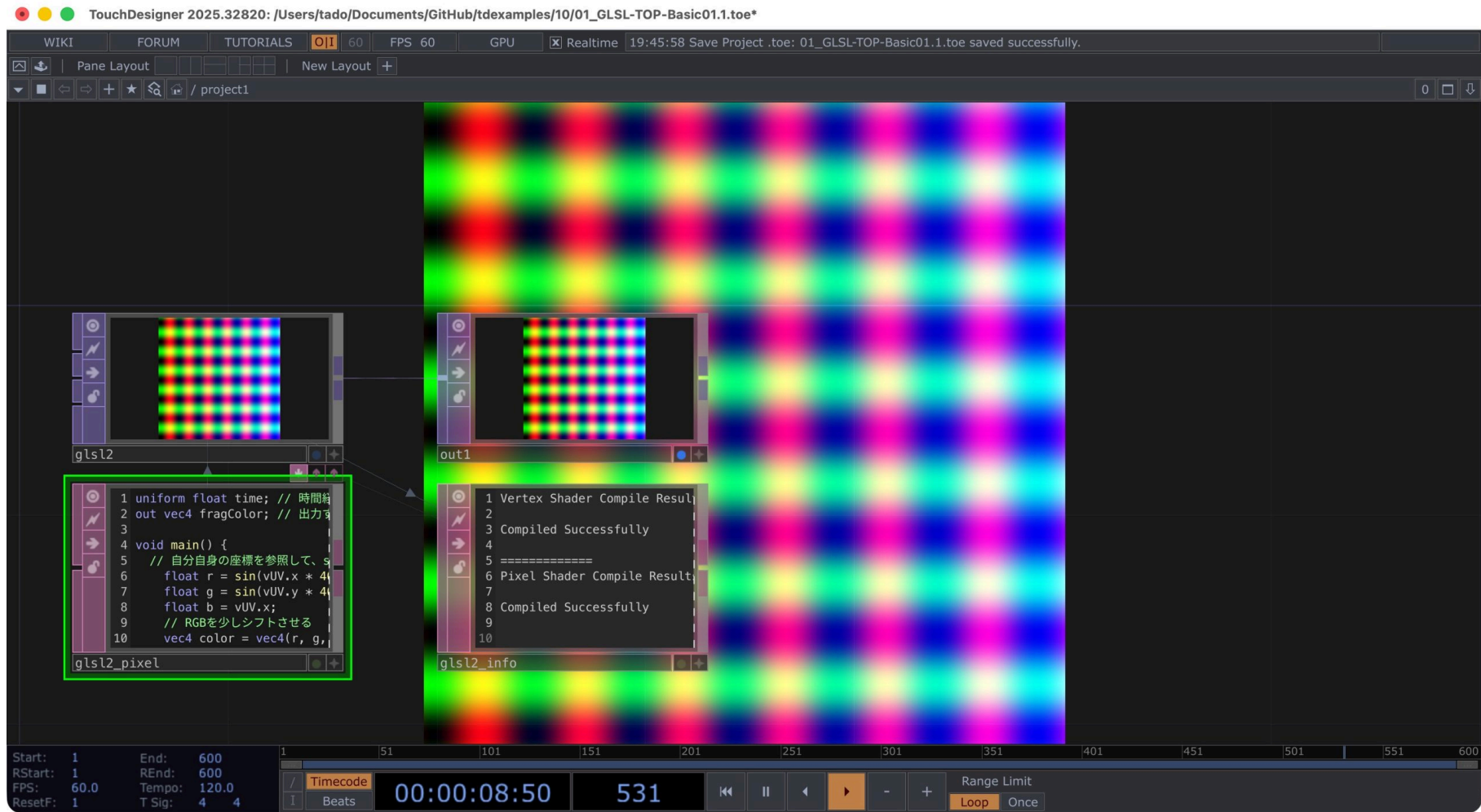
- 時間経過で移動する波を描く
- ポイント：自分自身の座標を参照 (vUV.x, vUV.y)
- ポイント：時間経過を取得するためのuniform変数 time
- 自分自身の位置 (vUV) と経過時間 (time) を使って、sin関数で移動する波を描く

移動する波

```
uniform float time; // 時間経過の値を取得するためのuniform変数
out vec4 fragColor; // 出力する色を格納する変数

void main() {
    // 自分自身の座標を参照して、sin関数で移動する波を描く
    float r = sin(vUV.x * 40 - time * 5.0) * 0.5 + 0.5;
    float g = sin(vUV.y * 40 - time * 5.0) * 0.5 + 0.5;
    float b = vUV.x;
    // RGBを少しシフトさせる
    vec4 color = vec4(r, g, b, 1.0);
    // 生成された色を出力する
    fragColor = TDOutputSwizzle(color);
}
```

移動する波が描けた!



ダウンロード: [GLSL TOP基本: 移動する波](#)

前回の復習 2: 広がる同心円

- 時間経過で広がる同心円を描く
- ポイント：中心点からの距離を計測してその値で形を描く
- GLSLで距離を測る関数 → `length()`

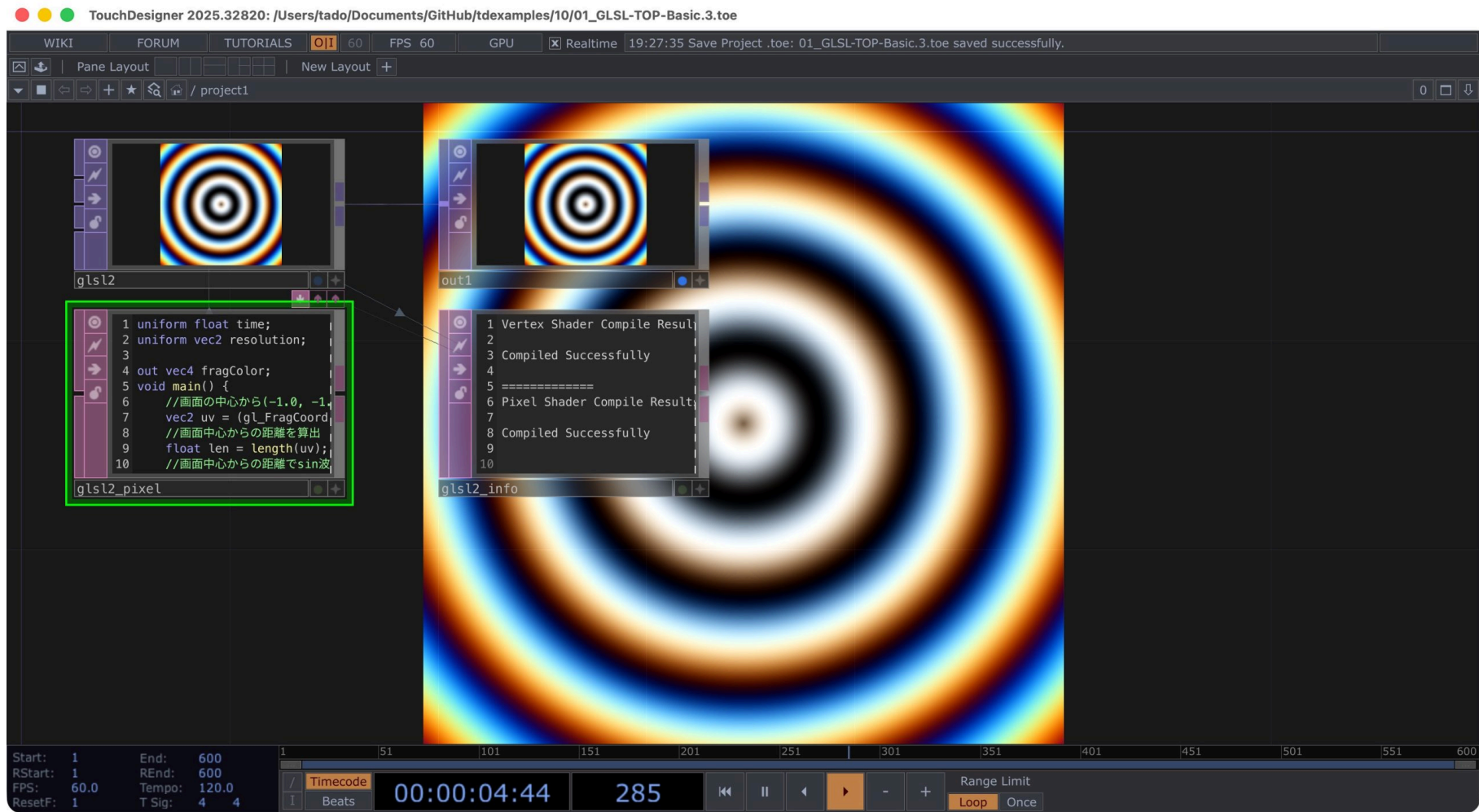
```
float len = length(startPos, endPos);
```

拡がる同心円

```
uniform float time;
uniform vec2 resolution;

out vec4 fragColor;
void main() {
    //画面の中心から(-1.0, -1.0)から(1.0, 1.0)の範囲の座標に変換し縦横の比率を揃える
    vec2 uv = (vUV.xy * 2.0 - resolution) / vec2(resolution.x, resolution.x);
    //画面中心からの距離を算出
    float len = length(uv);
    //画面中心からの距離でsin波を生成し同心円状の波に
    //RGBを少しシフトさせる
    float r = sin(len * 20 - time * 5.0) * 0.5 + 0.5;
    float g = sin(len * 21 - time * 5.0 - 0.2) * 0.5 + 0.5;
    float b = sin(len * 22 - time * 5.0 - 0.4) * 0.5 + 0.5;
    vec4 color = vec4(r, g, b, 1.0);
    fragColor = TDOutputSwizzle(color);
}
```

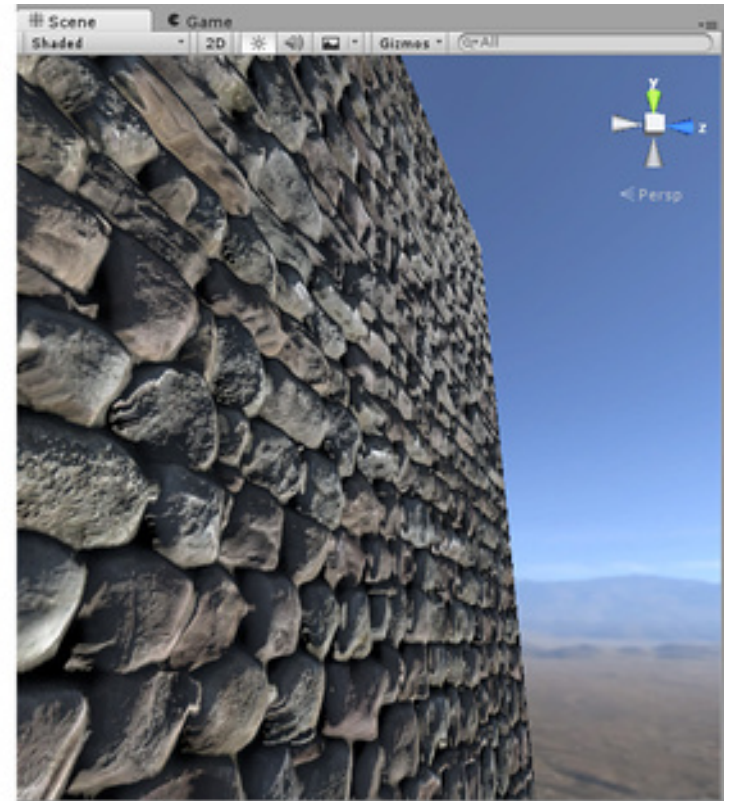
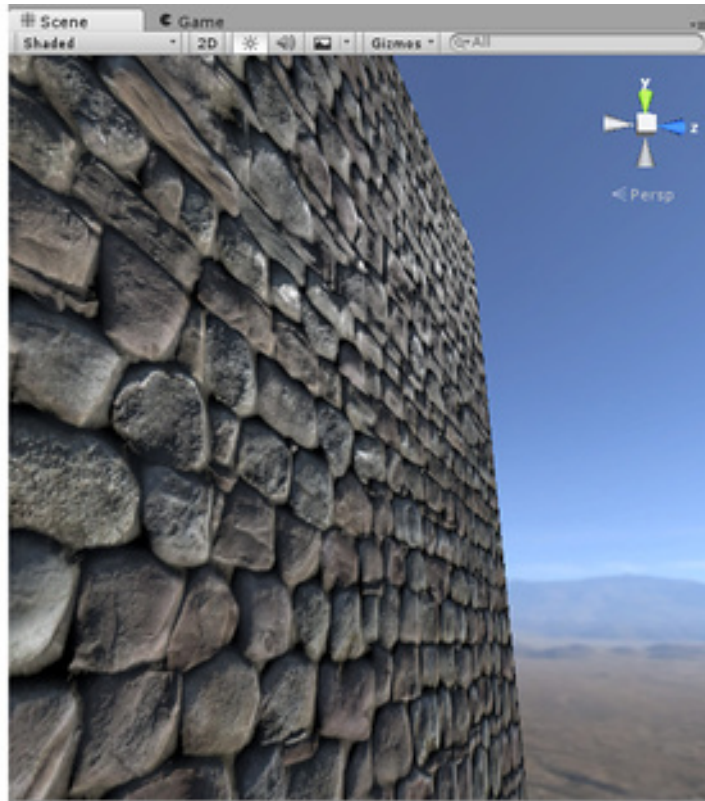

中心から広がる同心円が描けた!



ダウンロード: [GLSL TOP基本: 広がる同心円](#)

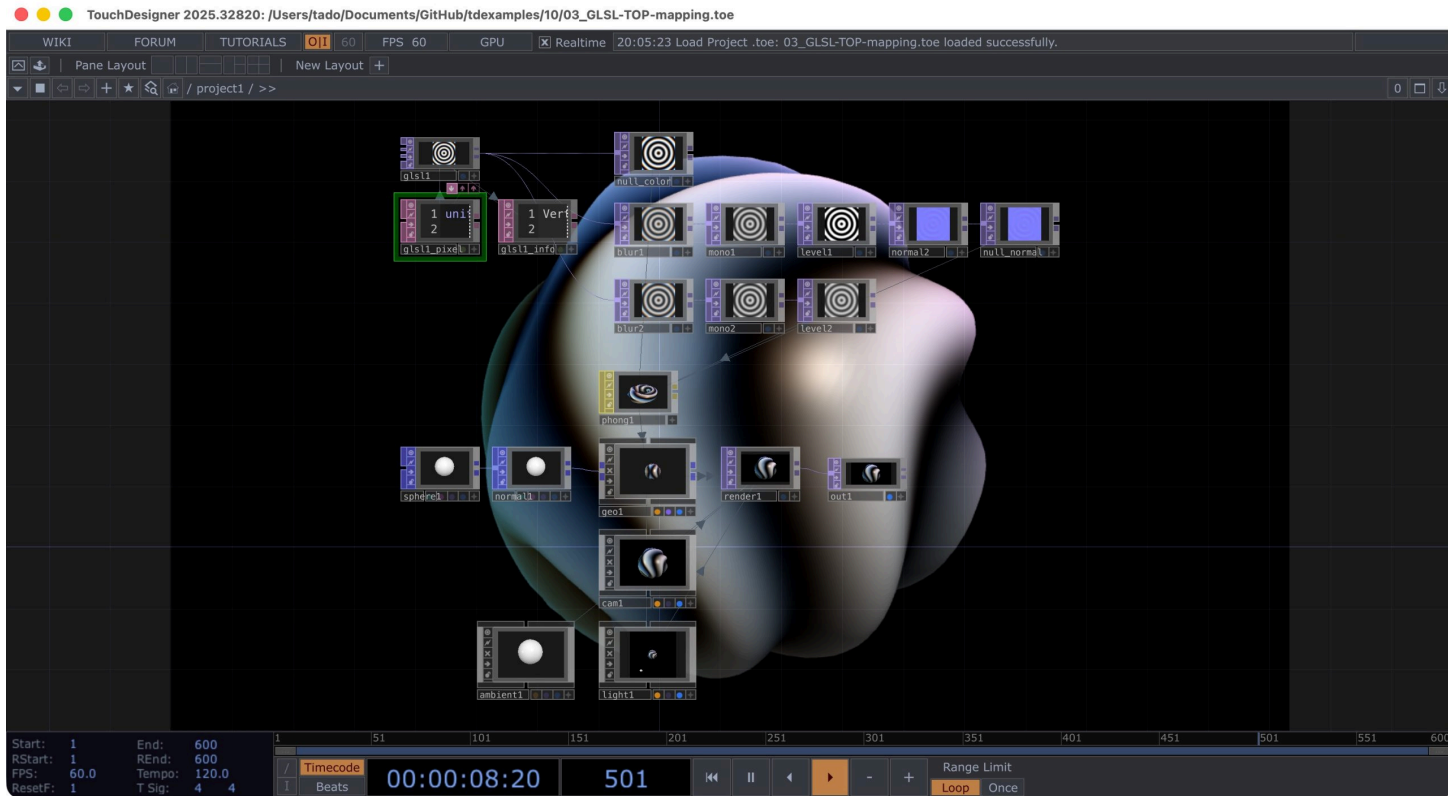
前回の復習 3: GLSLで生成されたイメージで凸凹を表現

- 同心円のアニメーションのパターンを用いて凸凹を生成
- ハイトマップ（Height Map）という手法を用いている
- ゲームエンジンなどで使用されている手法



- GLSLで生成されたイメージを使って凸凹を表現する
- 球体をPOPでレンダリングして、Phong MATを適用
- Phong MATで以下のように設定する
 - 出力されたTextureをColor Mapとしてマッピング
 - 出力されたTextureをNormal Mapに変換して、バンプマッピングを行う
 - 出力されたTextureをBlur TopやLevel TOPで加工した後ハイトマップとして使用する

バンプマッピング + ハイトマップで凸凹を表現!



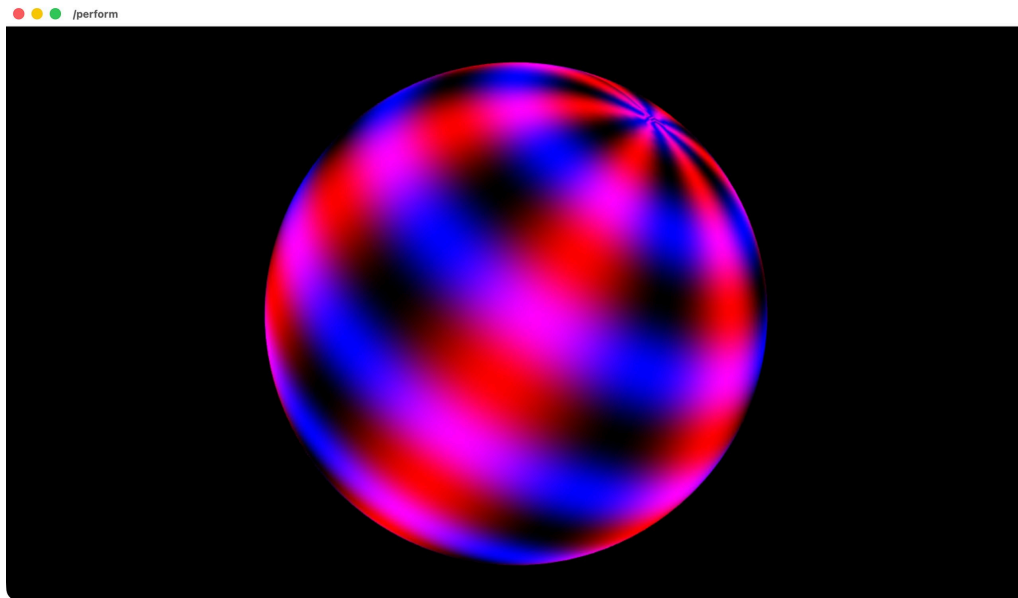
ダウンロード: [GLSL TOP基本: バンプマッピングとハイトマッピングで凸凹表現](#)

先週の復習: ここまでのまとめ

- 画面上のピクセル単位での色を計算しているのがFragment Shader
- 画面の全てのピクセルにプログラムが埋め込まれているイメージ
- 最終出力する色は、gl_FragColorに格納する
- 自分自身の座標を参照するための変数がvUV (vUV.x, vUV.y)
- 時間経過を取得するためには、uniform変数で外部から時間経過の値を渡す

先週の復習: GLSL TOPを立体に適用するということは？

- Fragment Shaderは、(vUV.x, vUV.y) で自分自身の位置を取得
- 奥行 (z) の情報は参照できない
- あくまで、平面のテクスチャを球面に貼り付けた状態
- 奥行情報も参照するには？ → GLSL MATを使う！



ダウンロード: [GLSL TOP: 球面に平面的に波を描く](#)

本日の内容: GLSL MATを使ってみる

- TouchDesignerでShader (GLSL) を使ってみる、応用編
- GLSL MATを使ってみる
- (x, y) 座標だけでなく、奥行 (z) も参照できるようになる
- Fragment Shaderに加えて、Vertex Shaderも編集

GLSL MATの基本構造

- GLSL MATを配置すると、3つのDATが自動で生成される
 - Vertex Shader用のDAT
 - Pixel Shader用のDAT
 - コンソール (エラー出力など) 用のDAT
- デフォルトのVertex ShaderとPixel Shaderを解析してみる

GLSL MATのVertex Shaderの基本コード

```
void main()
{
    // ローカル座標を取得
    vec3 pos = TDPos();
    // 変換後の座標をgl_Positionに代入
    vec4 worldSpacePos = TDDeform(pos);
    // 変換後の座標をgl_Positionに代入
    // gl_Positionが頂点の最終的な位置を出力する
    gl_Position = TDWorldToProj(worldSpacePos);
}
```

GLSL MATのFragment Shaderの基本コード

```
out vec4 oFragColor; // 出力カラー
void main()
{
    // ピクセルが画面内に描画されるかどうかを判定する関数
    TDCheckDiscard();
    // 出力カラーを設定する
    vec4 color = vec4(1.0);
    // アルファ値を使ってピクセルの描画を制御する関数
    TDAlphaTest(color.a);
    // 最終的な出力カラーを設定する
    // oFragColorは、最終的に画面に描画されるピクセルの色を表す変数
    oFragColor = TDOutputSwizzle(color);
}
```

Vertex ShaderからFragment Shaderに値を渡す

- Vertex Shaderで計算した値をFragment Shaderに渡すことができる (varying変数を使用)
- Vertex Shader の out 変数と、それを受け取る Fragment Shader の in 変数のペア

例:

```
// Vertex Shader側  
out vec3 localPos; // ローカル座標をFragment Shaderに渡す
```

```
// Fragment Shader側  
in vec3 localPos; // Vertex Shaderから渡されたローカル座標を受け取る
```

Vertex Shaderで取得したローカル座標を使って、Fragment Shaderで色を計算する

- Vertex Shader側で `TDPos()` を使ってローカル座標を取得し、それを `out` 変数に格納
- Fragment Shader側で `in` 変数を使ってローカル座標を受け取り、色を計算する
- 奥行 (`z`) の情報も参照できるようになる
- `x, y, z` でそれぞれ `r, g, b` の波を生成して、時間経過で移動する波を描いてみよう!

Vertex Shader

```
// ローカル座標をPixel Shaderに渡すための出力変数
out vec3 localPos;

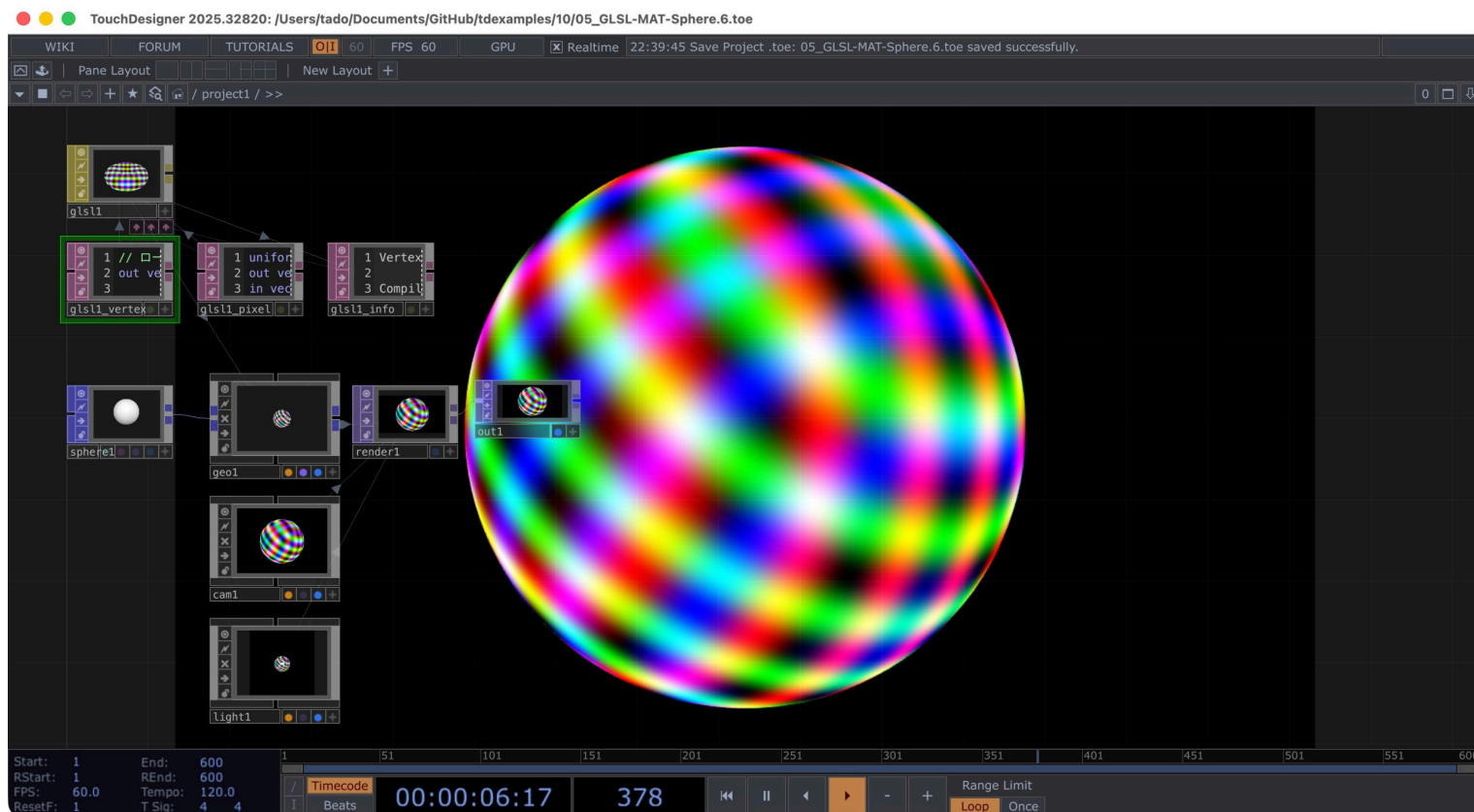
void main() {
    // ローカル座標を取得
    vec3 pos = TDPos();
    // ローカル座標をPixel Shaderに渡す
    vec4 worldSpacePos = TDDeform(pos);
    // 変換後の座標をgl_Positionに代入
    gl_Position = TDWorldToProj(worldSpacePos);
    // ローカル座標をPixel Shaderに渡す
    localPos = pos;
}
```

Fragment Shader

```
uniform float time; // 時間経過の値を取得するためのuniform変数
out vec4 oFragColor; // 出力する色を格納する変数
in vec3 localPos; // ローカル座標を取得するための入力変数

void main() {
    // ピクセルが画面内に描画されるかどうかを判定する関数
    TDCheckDiscard();
    // ローカル座標を使って色を計算する
    float r = sin(localPos.x * 20 - time * 5.0) * 0.5 + 0.5;
    float g = sin(localPos.y * 20 - time * 5.0) * 0.5 + 0.5;
    float b = sin(localPos.z * 20 - time * 5.0) * 0.5 + 0.5;
    float a = 1.0;
    vec4 color = vec4(r, g, b, a);
    // アルファ値を使ってピクセルの描画を制御する関数
    TDAlphaTest(color.a);
    // 最終的な出力カラーを設定する
    oFragColor = TDOutputSwizzle(color);
}
```

x, y, z のそれぞれの座標を使って、r, g, b の波を生成して、球体の表面に沿って色が変わるアニメーションが描画される



ダウンロード: [GLSL MAT: 球面に立体的に波を描く](#)

3D空間上の中心からの距離で色を変化させる

- GLSL TOPの時と同じように、length() 関数を使って、3D空間上の中心からの距離を計算可能
- その距離を使って、色を変化させることができる
- 球体 (Sphere POP) をNoise POPで変形させて、中心からの距離で色を変化させるGLSL MATを作ってみよう!

Vertex Shaderは変更なし

```
// ローカル座標をPixel Shaderに渡すための出力変数
out vec3 localPos;

void main() {
    // ローカル座標を取得
    vec3 pos = TDPos();
    // ローカル座標をPixel Shaderに渡す
    vec4 worldSpacePos = TDDeform(pos);
    // 変換後の座標をgl_Positionに代入
    gl_Position = TDWorldToProj(worldSpacePos);
    // ローカル座標をPixel Shaderに渡す
    localPos = pos;
}
```

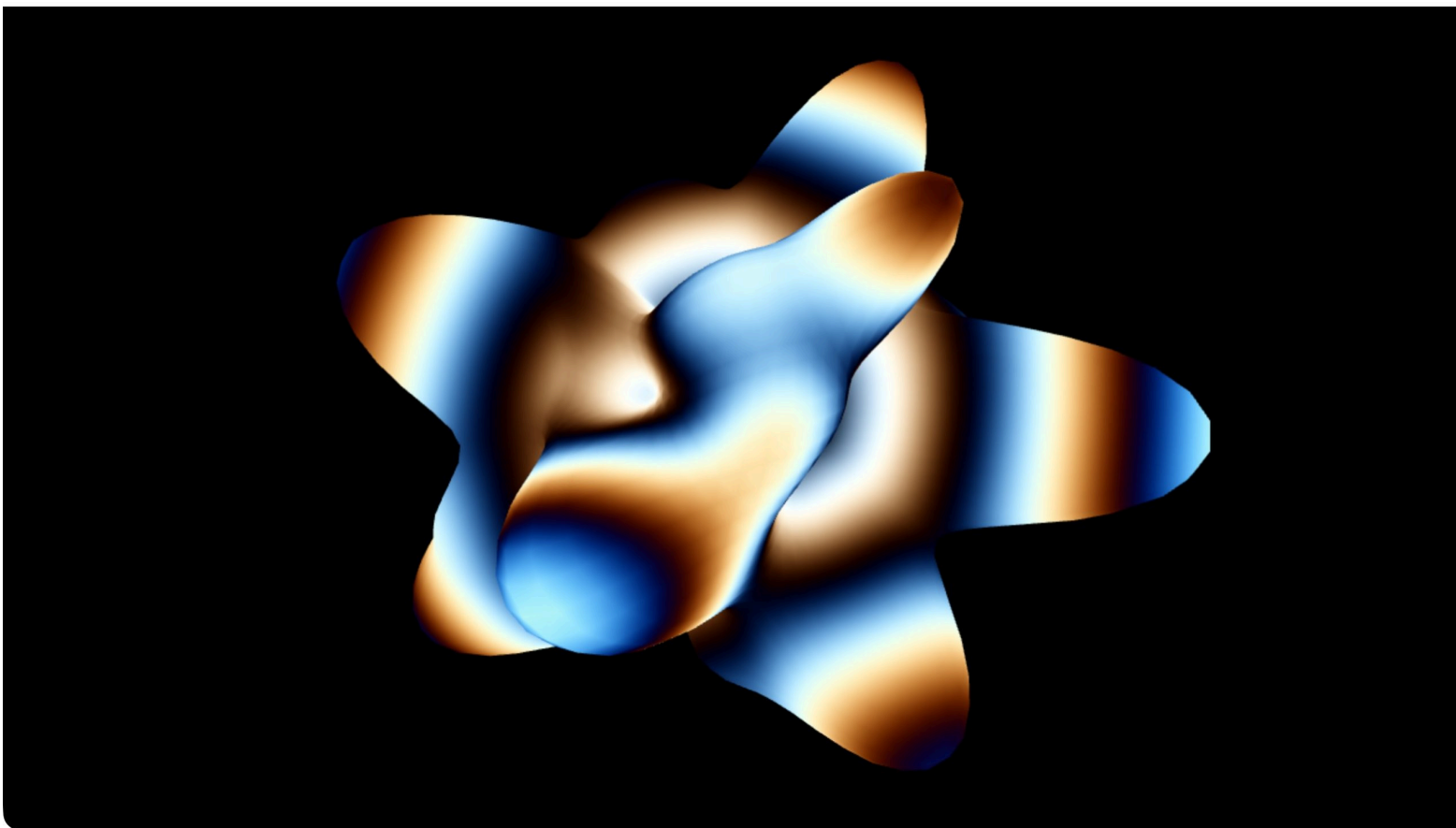
Fragment Shader

```
uniform float time; // 時間経過の値を取得するためのuniform変数
out vec4 oFragColor; // 出力する色を格納する変数
in vec3 localPos; // ローカル座標を取得するための入力変数

void main() {
    // ピクセルが画面内に描画されるかどうかを判定する関数
    TDCheckDiscard();
    // ローカル座標の中心からの距離を計算する
    float len = length(localPos);
    // 中心からの距離を使って、r, g, b の波を生成する
    float r = sin(len * 10.0 - time * 3.0) * 0.5 + 0.5;
    float g = sin(len * 10.4 - time * 3.0) * 0.5 + 0.5;
    float b = sin(len * 10.8 - time * 3.0) * 0.5 + 0.5;
    float a = 1.0;
    vec4 color = vec4(r, g, b, a);
    // アルファ値を使ってピクセルの描画を制御する関数
    TDAlphaTest(color.a);
    // 最終的な出力カラーを設定する
    oFragColor = TDOutputSwizzle(color);
}
```

中心からの距離で色が変わ!

● ● ● /perform*



ダウンロード: [GLSL MAT: 中心からの距離で色を変化させる](#)

アルファ値を使って、ピクセルの描画を制御する

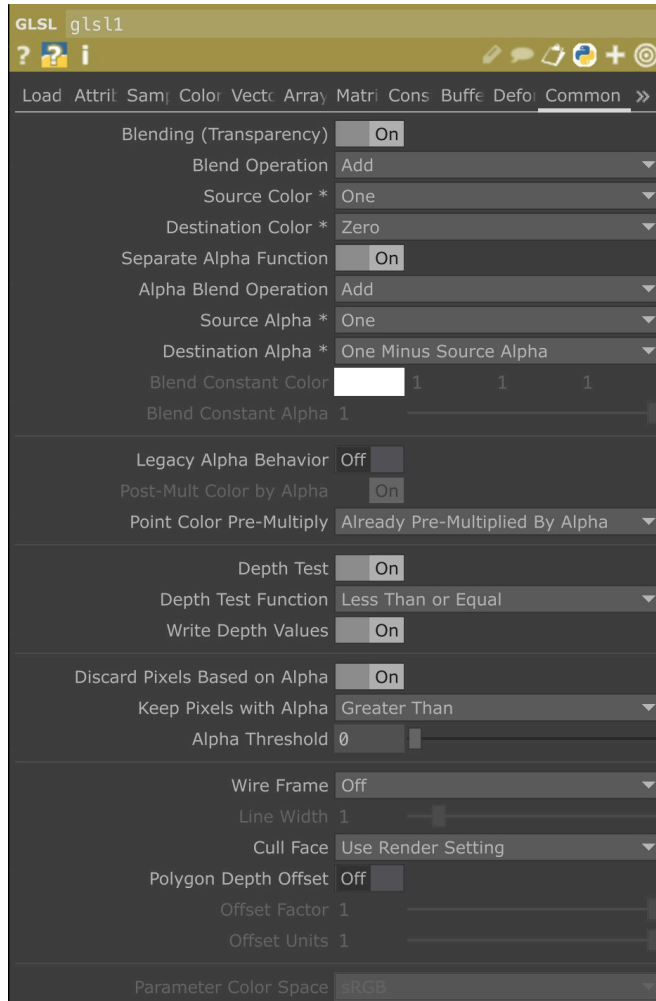
- TDAlphaTest() 関数を使うことで、アルファ値を使ってピクセルの描画を制御することができる
- 閾値より小さいアルファ値のピクセルは描画されないようにする
- 面の一部が透明になるような表現が可能になる

Fragment Shaderの例

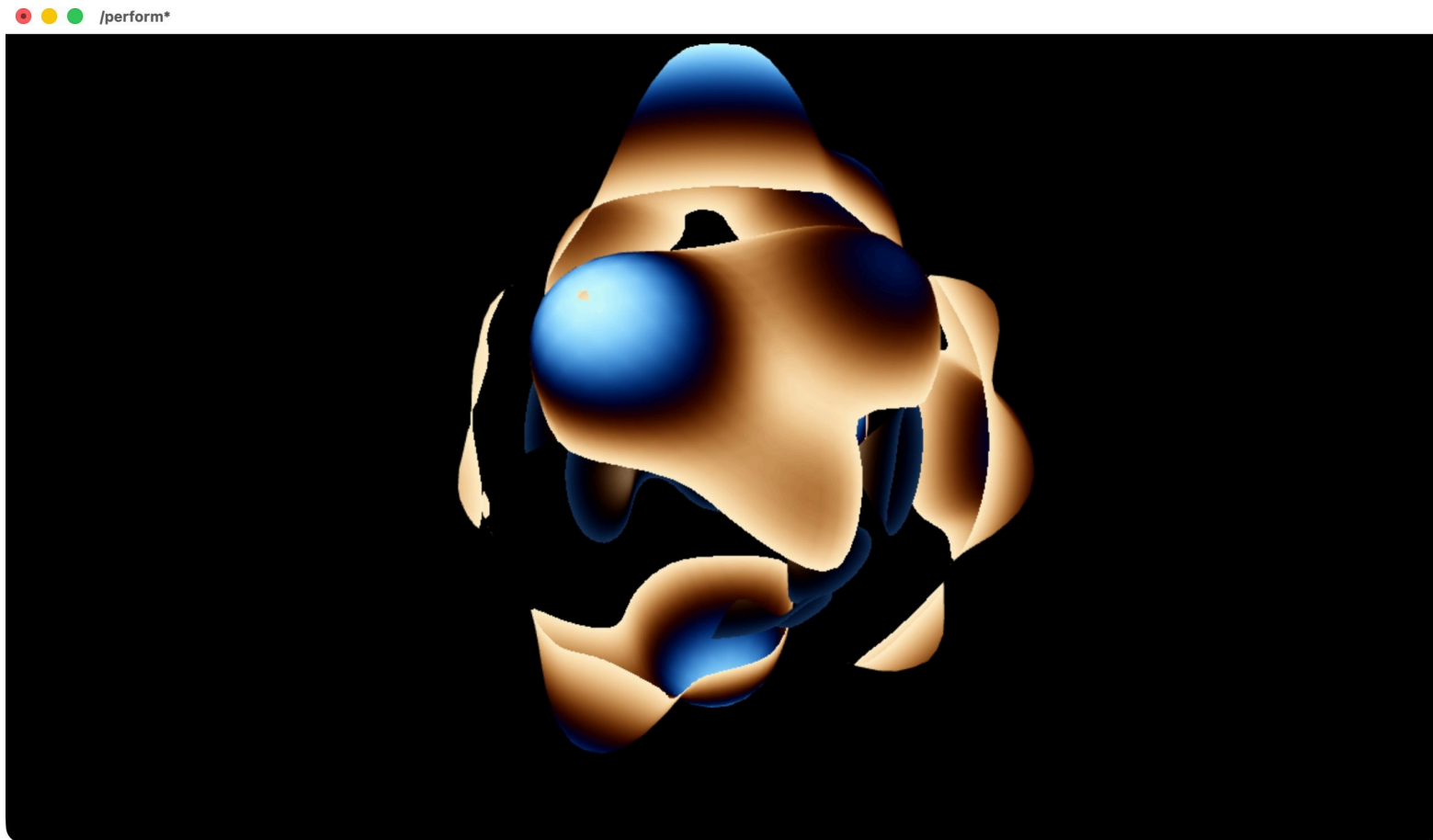
```
uniform float time; // 時間経過の値を取得するためのuniform変数
out vec4 oFragColor; // 出力する色を格納する変数
in vec3 localPos; // ローカル座標を取得するための入力変数

void main() {
    // ピクセルが画面内に描画されるかどうかを判定する関数
    TDCheckDiscard();
    // ローカル座標の中心からの距離を計算する
    float len = length(localPos);
    // 中心からの距離を使って、r, g, b の波を生成する
    float r = sin(len * 10.0 - time * 3.0) * 0.5 + 0.5;
    float g = sin(len * 10.4 - time * 3.0) * 0.5 + 0.5;
    float b = sin(len * 10.8 - time * 3.0) * 0.5 + 0.5;
    // アルファ値も同様に距離と時間を使って波を生成する
    float a = sin(len * 12.0 - time * 3.0);
    vec4 color = vec4(r, g, b, a);
    // アルファ値を使ってピクセルの描画を制御する関数
    TDAlphaTest(color.a);
    // 最終的な出力カラーを設定する
    oFragColor = TDOutputSwizzle(color);
}
```

GLSL MATのCommonタブで、アルファ値による透明度の描画を設定する



アルファ値を使って、面の一部が透明になるような表現が可能になった!



ダウンロード: [GLSL MAT: アルファ値を使ってピクセルの描画を制御する](#)

Phong MATのShaderを編集してオリジナルの表現を作る

- GLSL MATによる描画 → 陰影がつかない
- Phong MATのShaderを編集することで、陰影がついた表現が可能になる
- Phong MATをGLSL MATに変換して、Vertex ShaderとFragment Shaderを編集することで、オリジナルの表現を作ることができる
- Phong MATのRGBタブの最下部の「Output Shader...」の横の「Output」ボタンを押す
→ GLSL MATに変換される

生成されたFragment Shaderに以下のコードを追加

```
// ... (前略) ...

void main() {

    // ... (中略) ...

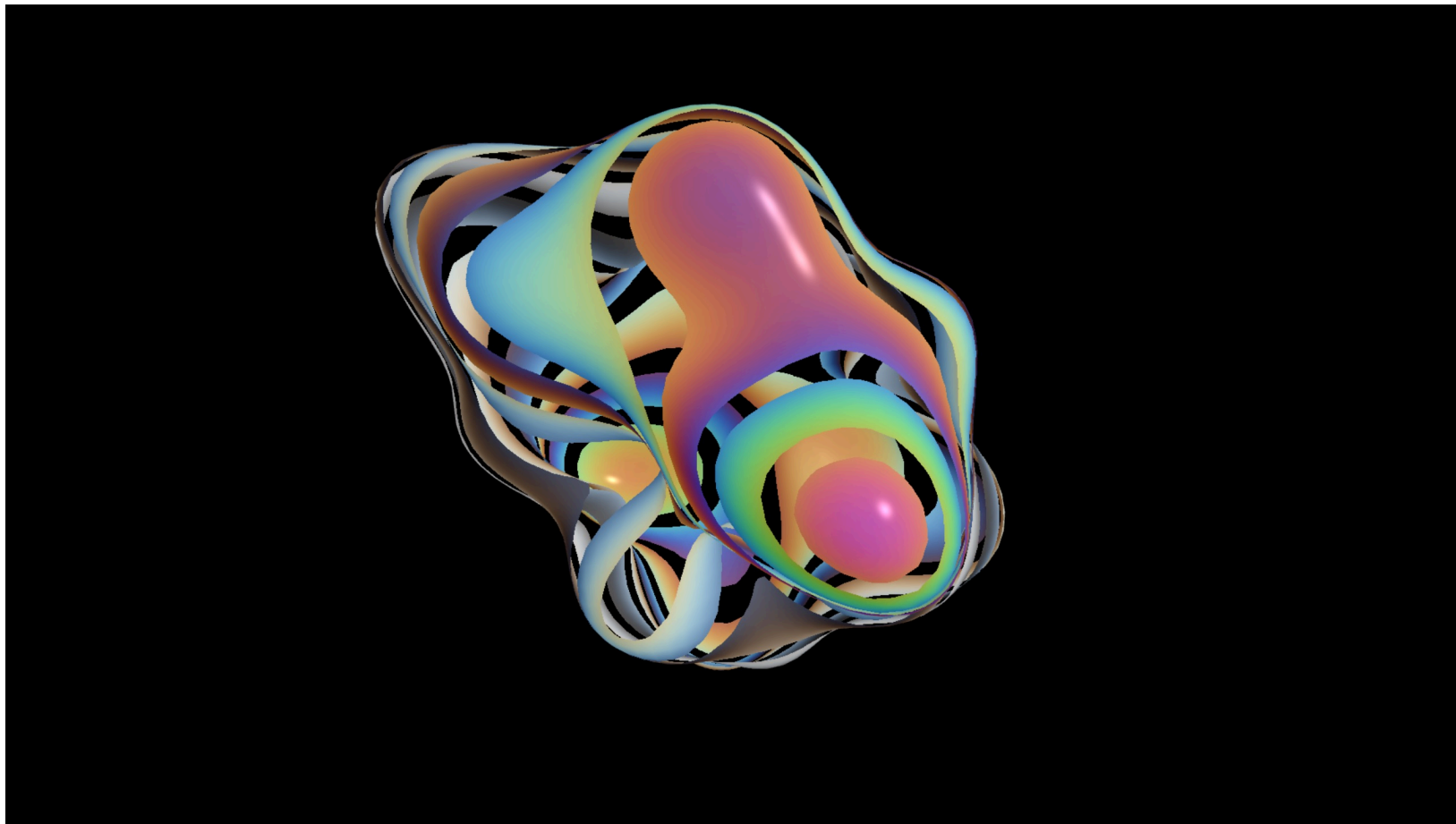
    // --- 以下の部分を追記することで、オリジナルな効果を生みだしている ---
    float r = sin(iVert.worldSpacePos.z * 20.0) * 0.5 + 0.5;
    float g = sin(iVert.worldSpacePos.z * 22.0) * 0.5 + 0.5;
    float b = sin(iVert.worldSpacePos.z * 24.0) * 0.5 + 0.5;
    float a = sin(iVert.worldSpacePos.z * 43.0) + 0.5;

    finalColor.r = (finalColor.r + r) / 2.0;
    finalColor.g = (finalColor.g + g) / 2.0;
    finalColor.b = (finalColor.b + b) / 2.0;
    finalColor.a = a;
    // --- ここまでが追記部分 ---

    // ... (中略) ...

}
```

Phong MATの陰影に加えて、オリジナルの色の変化が加わった表現が可能になった!



ダウンロード: [Phong MATのShaderを編集してオリジナルの表現を作る - 球体](#)

Noiseで変形する元のPOPをSphereからGridに変更しても面白い!



ダウンロード: [Phong MATのShaderを編集してオリジナルの表現を作る - グリッド](#)